

COS 214 Practical 4 — TaskForge

Meridian Records: Hierarchical Work Processing

Members:

Kelly Rademeyer: u25571878

Tebogo Mokgwatsane: u25042239

Lebogang Nxumalo: u05175552

1. System Description

Meridian Records is a music production company managing a catalogue of artist projects. The system models the company's work hierarchy: a **RecordLabel** contains several **ArtistProjects** (albums), each of which contains one or more **RecordingSessions**, each of which contains individual **Tracks**. This gives three levels of composite nesting below the root, with Track as the only leaf, satisfying the practical's structural requirement.

Each Track has a genuine production lifecycle — Demo, Recording, Mixing, Mastering, and Released — and its cost and estimated completion time change meaningfully as it moves through these stages. Tracks can also be given optional, stackable extras at runtime: Explicit Content labelling, Rush Order handling, a Featured Artist credit, or a Remix pass, in any combination.

The system supports two independent ways of traversing the same hierarchy: a full-catalogue traversal used for company-wide royalty and cost reporting, and a selective traversal that returns only the Tracks currently sitting at a given lifecycle stage, used to generate a single engineer's work queue.

2. UML Class Diagram

(attached separately)

GoF Participant

Pattern	Participant role	Class(es)
Composite	Component	WorkComponent
Composite	Composite	RecordLabel, ArtistProject, RecordingSession
Composite	Leaf	Track
State	Context	Track
State	State	TrackState

State	ConcreteState	DemoState, RecordingState, MixingState, MasteringState, ReleasedState
Decorator	Component	WorkComponent
Decorator	ConcreteComponent	Track
Decorator	Decorator	TrackDecorator
Decorator	ConcreteDecorator	ExplicitContentDecorator, RushOrderDecorator, FeaturedArtistDecorator, RemixDecorator
Iterator	Iterator	Iterator (first/next/isDone/currentItem)
Iterator	ConcreteIterator	FullCatalogIterator, PendingStageIterator
Iterator	Aggregate	WorkComponent (createIterator hook)

3. Design and Ownership Rationale

Design Rationale — Composite Hierarchy (Person A)

Why Composite fits Meridian Records

Meridian Records naturally organises work as a nested part-whole structure: a label contains multiple artist projects, each project contains multiple recording sessions, and each session contains multiple tracks. Reporting tasks in this domain — for example, "what is the total production cost of this label?" — need to aggregate over every level of this nesting without the caller needing to know how deep the structure goes or how many items exist at each level.

Composite solves this directly: WorkComponent defines one interface shared by both the single item (Track) and every level of grouping (RecordingSession, ArtistProject, RecordLabel). A call to getTotalCost() on the root RecordLabel recurses transparently through every project, session, and track beneath it, without any type-checking or special-casing of which kind of node is being visited. This satisfies Rule 8 (no giant switch/if-else chains standing in for the pattern) and keeps main.cpp decoupled from the concrete shape of the hierarchy.

An alternative considered was giving RecordLabel and ArtistProject separate, non-uniform interfaces from Track (e.g. a getAggregateCost() method only on composites). This was rejected because it reintroduces the exact type-checking problem Composite exists to avoid — client code would need to ask "is this a leaf or a composite?" before knowing which method to call.

1. Ownership and destruction policy

Every composite (RecordLabel, ArtistProject, RecordingSession) owns the WorkComponent* children it holds in its private container. Each composite's destructor iterates over its children and calls delete on each one. Because this is recursive by construction (a composite's children may themselves be composites with their own destructors), deleting the root RecordLabel triggers a full cascade: every ArtistProject, every RecordingSession, and every Track beneath it is destroyed automatically.

This was chosen over alternatives such as shared ownership (e.g. shared_ptr) or caller-managed lifetimes for two reasons:

2. **Simplicity of reasoning.** A single, consistent rule — "the parent owns and deletes its children" — is straightforward to verify by inspection and holds at every level of the hierarchy without exception.
3. **Matches the domain.** A Track genuinely has no independent existence outside the RecordingSession it was recorded in; deleting the session should reasonably delete its tracks too. There is no use case in this domain for a track to outlive the session that owns it.

WorkComponent declares a virtual destructor, satisfying Rule 4, so deleting through a base-class pointer (e.g. WorkComponent* root = label; delete root;) correctly invokes each derived class's destructor rather than only the base class's.

Satisfying Rule 7 (no container exposure)

The children container in every composite is declared private. Client code (main.cpp) has no way to obtain it directly. All traversal is done through createIterator(), which returns an Iterator* exposing only hasNext()/next() — the concrete storage mechanism (std::vector) is never visible outside the composite class that owns it.

State/Decorator ownership:

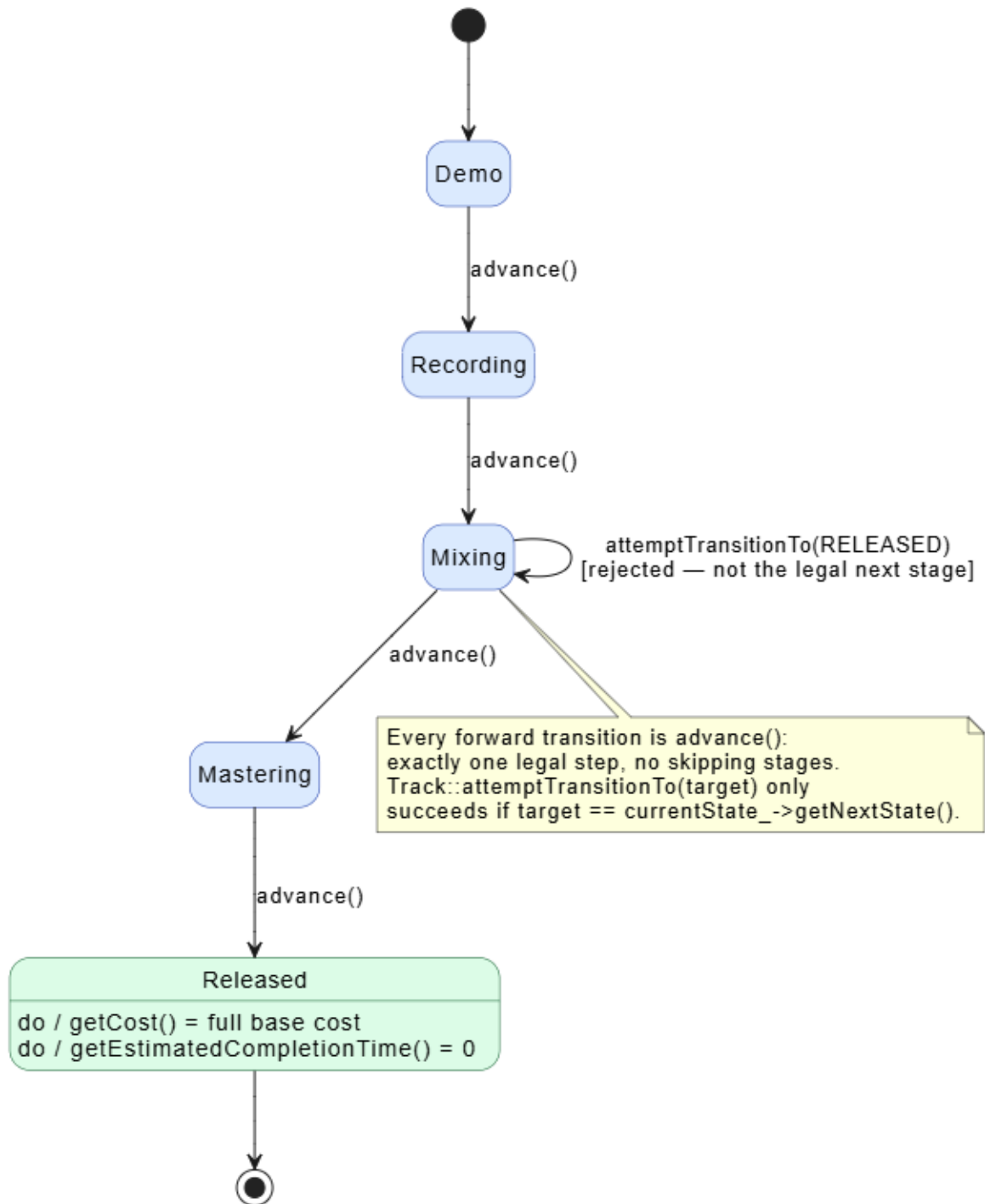
Track owns exactly one TrackState* at a time (field currentState_), deleted in Track's destructor and replaced — with the old state deleted — on every legal transition. TrackDecorator owns the WorkComponent* it wraps and deletes it in its own destructor, so deleting the outermost decorator in a stack correctly cascades all the way down to the real Track underneath it.

4. Object Diagram and State Diagram

4.1 Object Diagram

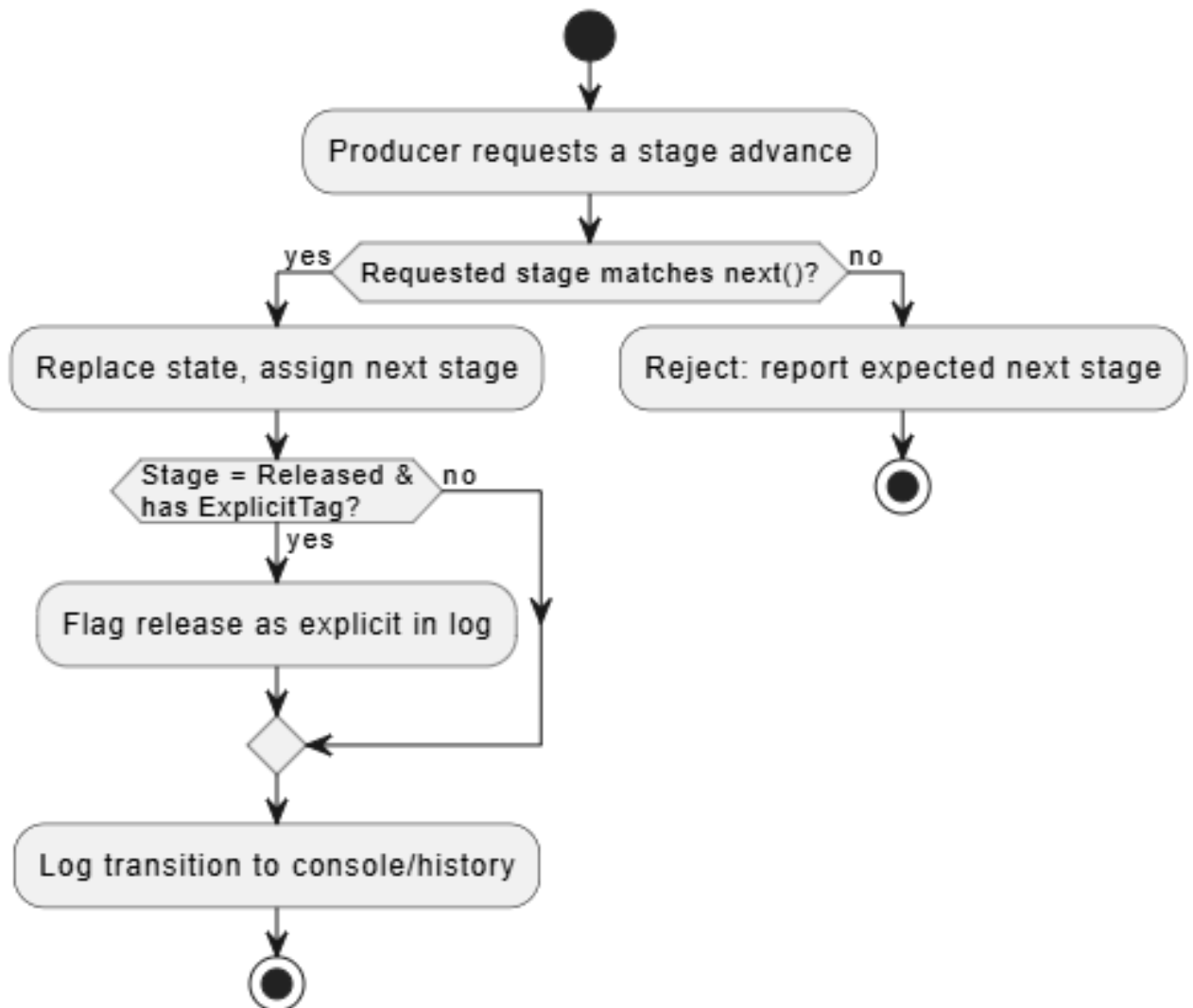
(attached separately)

4.2 State Diagram — Track Lifecycle

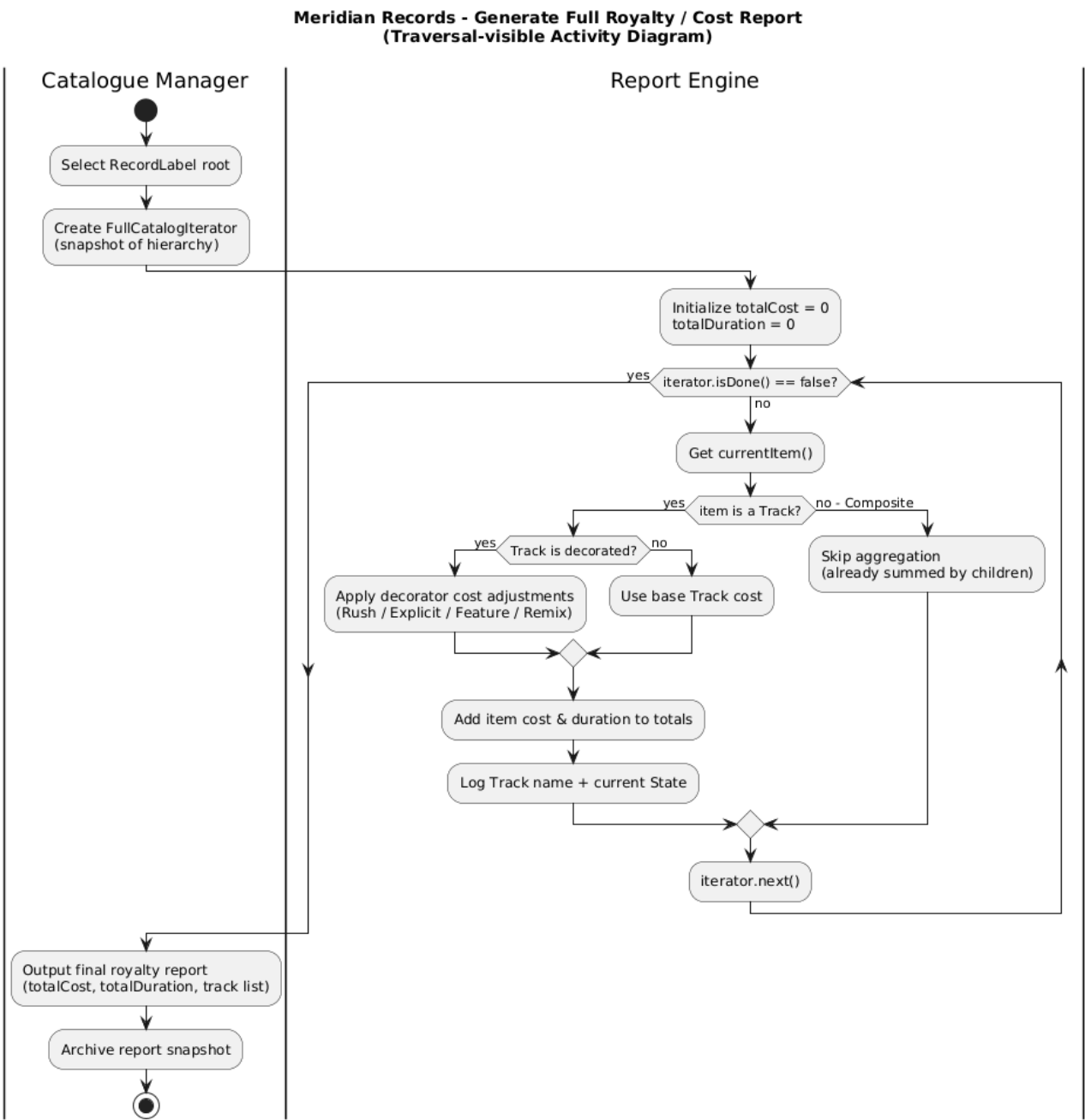


5. Activity Diagrams

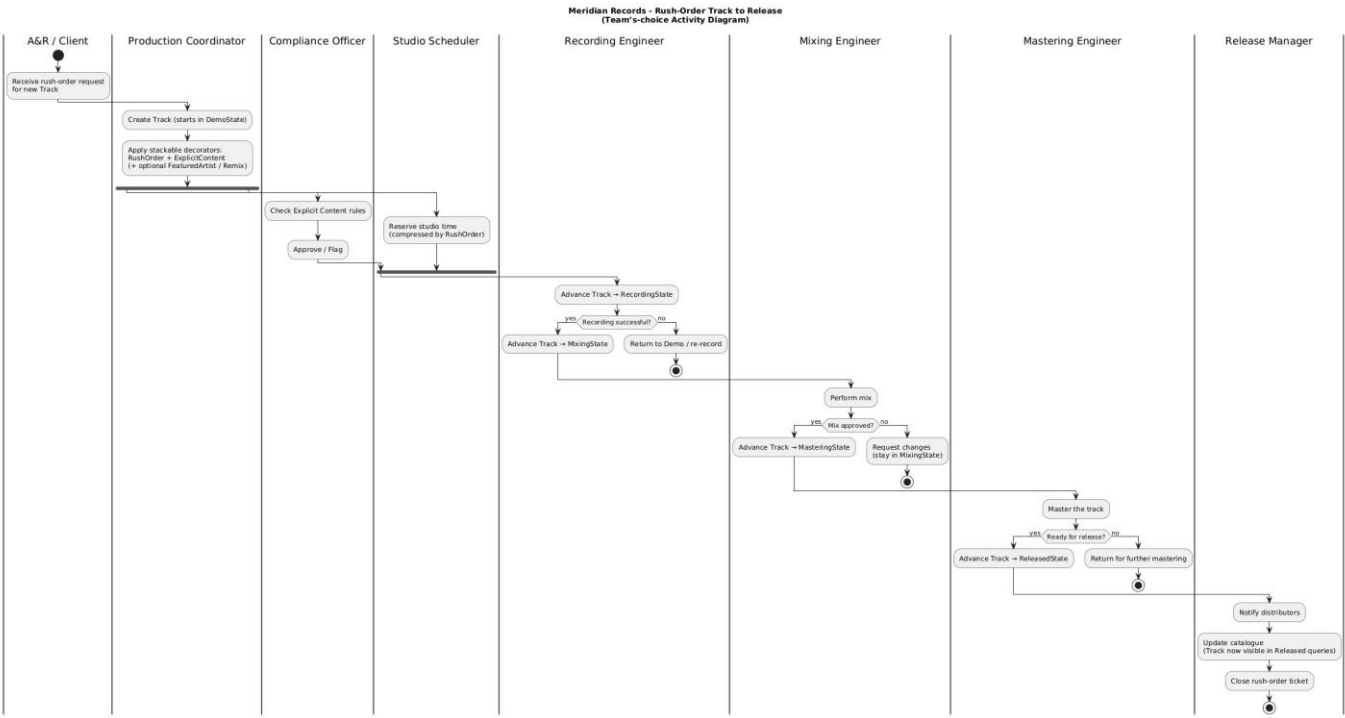
5.1 Lifecycle-conditional Activity Diagram



5.2 Traversal-visible Activity Diagram



5.3 Team's-choice Activity Diagram



6. Traversal-Modification Policy and Other Design Decisions

6.1 Traversal-modification policy: Snapshot

Snapshot - Both `FullCatalogIterator` and `PendingStageIterator` capture the set of objects they will visit at the moment they are constructed.

- Structural changes (add / remove / move of a `WorkComponent`) that occur after an iterator has been created do not appear in that iterator.
- State changes on already-captured objects are visible when the object is later queried (because we store pointers, not deep copies of state), but the membership of the iteration is fixed.
- This matches the common use-case of generating a report or a work-queue view that must remain stable for the duration of the processing run.

Decorator transparency to Composite/Iterator

`TrackDecorator` forwards `getChildrenForIteration()` and `matchesStage()` to its wrapped component, in addition to `getName()/getTotalCost()/getTotalDuration()/getStageName()`. This was not obvious at first — see the bug writeup in Section 7 — and is what allows a fully decorated `Track` (e.g. `RushOrderDecorator` wrapping `ExplicitContentDecorator` wrapping a `Track`) to be found correctly by either iterator and to report its true current stage, exactly as if it were an undecorated `Track`.

7. Engineering Investigation

7.1 Genuine bug: `TrackDecorator` breaking Iterator visibility

Symptom: A `Track` wrapped in one or more decorators and added to the hierarchy was not found by `PendingStageIterator`, even when the underlying `Track` was genuinely at the target stage.

Cause: `WorkComponent` provides default implementations of `getChildrenForIteration()` (empty) and `matchesStage()` (false) to support leaf classes that aren't composites or stateful. `TrackDecorator` originally forwarded `getName()/getTotalCost()/getTotalDuration()/getStageName()` to its wrapped component, but not these two — so a decorated `Track` silently inherited `WorkComponent`'s defaults instead of forwarding to the real `Track` underneath. It compiled and ran without error; the bug only became visible once a decorated `Track` was actually passed through an iterator.

Correction: Added explicit pass-through overrides to `TrackDecorator`:

```
getChildrenForIteration() override { return wrapped_ -> getChildrenForIteration(); } matchesStage(Stage target) override { return wrapped_ -> matchesStage(target); }
```

Verified with a targeted test that wraps a `Track` in three stacked decorators, advances it to `Mixing`, and confirms `PendingStageIterator` finds it correctly by name and reports the correct stacked name and adjusted cost.

Valgrind Evidence

Final Valgrind Evidence

Command used:

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
```

```
==48885== HEAP SUMMARY:
```

```
==48885==    in use at exit: 0 bytes in 0 blocks
```

```
==48885== total heap usage: 97 allocs, 97 frees, 77,121 bytes allocated
```

```
==48885==
```

```
==48885== All heap blocks were freed -- no leaks are possible
```

```
==48885==
```

```
==48885== For lists of detected and suppressed errors, rerun with: -s
```

```
==48885== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```